CIFOB-Bio project report: Evolution and extinction in GA models for a linear regression problem

07/06/2025

Inês Wemans 70144, João da Silva 69971, Ricardo Reis 70627

1.Introduction

Metaheuristic algorithms have become increasingly popular in recent years for solving complex real-life problems across diverse fields such as economics, engineering, and healthcare (Kumar *et al*, 2014). These algorithms are broadly categorized into single-solution and population-based approaches. While single-solution algorithms like simulated annealing risk getting trapped in local optima (Kumar et al, 2017), population-based metaheuristics, such as Genetic Algorithms (GA) (Michalewicz, 1992), Particle Swarm Optimization (PSO) (Kennedy *et al*, 1995), and Ant Colony Optimization (ACO) (Dorigo *et al*, 2006), maintain diversity among solutions, enabling more efficient global exploration.

Among population-based algorithms, Genetic Algorithms stand out as one of the most widely adopted heuristics, drawing inspiration from the Darwinian theory of evolution (Michalewicz *et al*, 1996). Firstly, proposed by J.H Holland in 1992, GAs are based on key evolutionary principles such as selection, crossover, and mutation operators (Michalewicz *et al*, 1996). These algorithms simulate the natural selection process, where the fittest individuals are more likely to survive and reproduce (Holland, 1992; Fogel *et al*, 1996). Solutions are encoded in vectors of values and evaluated using a fitness function, guiding the algorithm towards better fitted individuals in the search space (Burak and Mengshoel, 2021; Barbareschi *et al*, 2022; Peng *et al*, 2021).

The performance of GAs depends on the balance between intensifications (exploiting the best-found solutions) and diversification (exploring new areas in the search space) (Kumar *et al*, 2014; Bonabeau *et al*, 1999). This is especially important when tailoring GAs to domain-specific problems, as it impacts their ability to avoid local optima and converge towards a global solution.

Both in the natural world and in GAs, extinction events have been a source of variability in populations and “creative solutions” thriving in evolution (Ye *et al*, 2021). In this way, implementing extinctions triggered by elite fitness stagnation with elitism can help introduce more variability for exploration when reaching a local optima – extinction – all while keeping the best fitted individuals around – elitism. Essentially, this creates alternating periods in the evolution process where exploration or exploitation is prioritized.

Parkinsons’s Disease (PD) is recognized as the second most prevalent neurodegenerative condition, with a report incident of 2-3% among individuals aged 65 and above. While it is primarily associated with motor symptoms such as involuntary tremors, PD also leads to a range of non-motor complications, including cognitive decline and emotional disturbances (Poewe *et al*, 2017).

Tracking the progression of this disease is mainly done using the Unified Parkinson's Disease rating scale (UPDRS) is the most adopted clinical instrument. However, its use relies on procedures that are often invasive, labor-intensive, and financially expensive. Previous works have used ML tools like genetic programming to predict UPDRS based on clinical data (Castelli, Vanneschi and Silva, 2014).

In this project, we aim to optimize a set of linear regression coefficients using GAs, to accurately predict UPDRS by combining these coefficients with clinical data. Therefore, the individuals we aim to evolve are lists of coefficients that we combine with clinical data to obtain ypred as is below:

$$ypred = \boldsymbol{Intecept}\ \boldsymbol{\alpha 0} + \boldsymbol{\alpha 1} \cdot x1 + \boldsymbol{\alpha 2} \cdot x2 + \ldots + \boldsymbol{\alpha M} \cdot xM$$

The optimization is guided by minimizing the fitness function, Root MeanSquared Error (RMSE). The RMSE value for an individual reflects how good the predictions made using the individual's coefficients are, on average, across all patients in the training data, when comparing, for each patient y pred and y real .

To achieve the goal of this project, we created five distinct models (Figure1). The first model follows a traditional evolutionary strategy, applying selection via tournament selection, and geometric crossover and mutation to iteratively evolve the population.

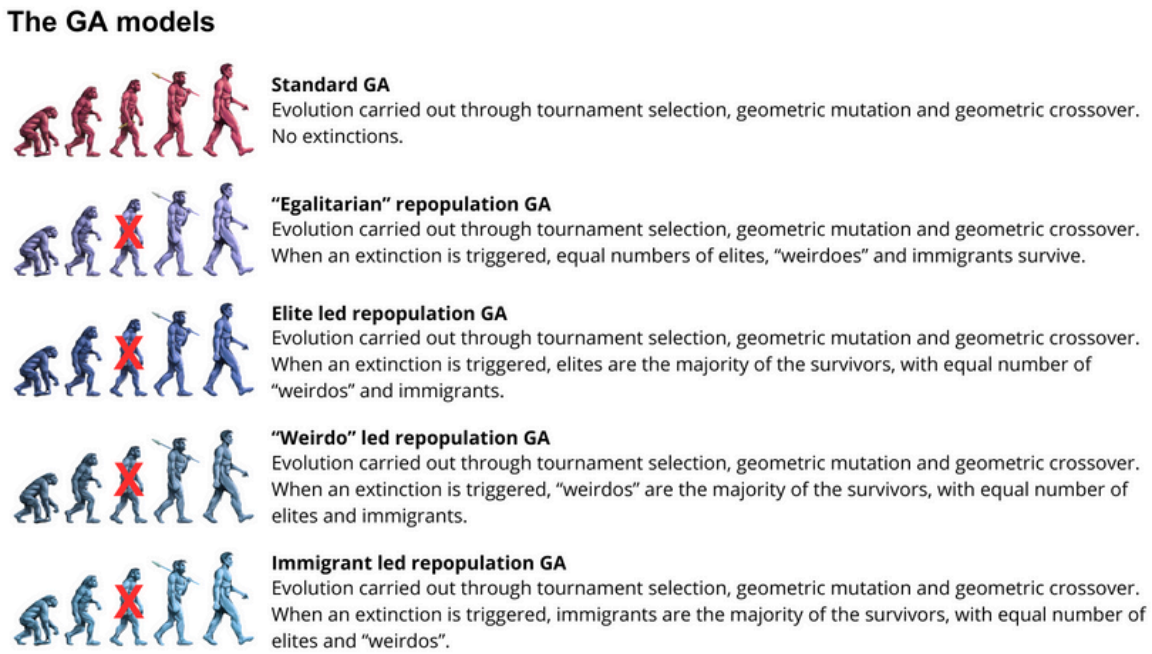

Figure1. The five GA models we created and evaluated in this project. From the top: 1. standard GA, 2. Egalitarian repopulation GA, 3. Elite led repopulation GA, 4. "Weirdo" led repopulation GA, 5. Immigrant led repopulation GA. The models that include extinctions are marked with a red "X".

We then developed a GA including extinction events triggered by elite fitness stagnation. Extinction events are characterized by removal of the vast majority of the population followed by repopulation with elites, "weirdos" and immigrants. Elites are the best fitted individuals in the extinction generation, "weirdos" are the most different individuals in the population (measured

geometrically) and immigrants are randomly generated individuals that are created in the same way that the individuals in the founder generation are created. The 4 extinction including GAs that we created differ in the repopulation strategy (Figure 1).

We have a model where repopulation is done with equal number of elite, "weirdo" and immigrant individuals, and three models where one of these types of individuals are mostly responsible for repopulation.

Ultimately, this work contributes to the ongoing efforts of leveraging metaheuristic algorithms for development more efficient, scalable, and non-evasive tools in the healthcare domain, specifically for monitoring and managing Parkinson Disease progression.

2.Methods

To develop, compare and test our five GA models, we followed the pipeline shown in Figure 2 that we will explain more in depth in this section of the report. We first began by developing a standard version of the GA without extinctions and a version of the GA with extinctions (steps 1.1 and 1.2 in Figure 2).

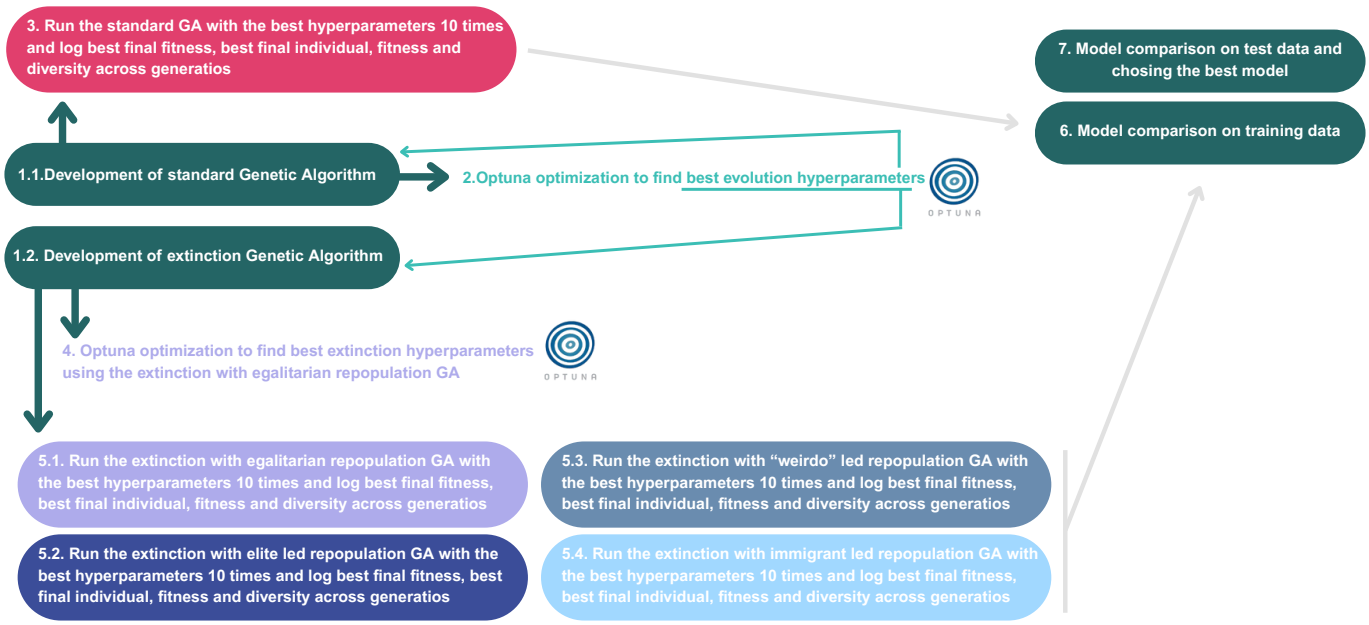

Figure 2. The development, hyperparameter tuning and performance analysis pipeline. Developement (steps 1.1. and 1.2.). Hyperparameter tuning (steps 2 and 4), Running the various models (steps 3, 5.1., 5.2., 5.3. and 5.4.). Performance analysis (steps 6 and 7).

Then, we used Optuna trials to optimize evolution hyperparameters towards minimizing a score that reflects both a good final fitness as well as steady evolution (step 2 in Figure 2). Those evolution hyperparameters were used to run the standard GA 10 times and the results of those runs (last elite individual (id and genotype), fitness of the last elite individual and population diversity per generation) were logged in txt files (step 3 in Figure 2).

Using the evolution hyperparameters, we used Optuna trials again to optimize extinction hyperparameters for the egalitarian repopulation GA (step 4 in Figure 2). With the optimized

evolution and extinction hyperparameters, we ran each version of the extinction GAs and logged 10 runs for each model in the same txt format described above for the standard GA (steps 5.1 to 5.4 in Figure 2).

Lastly, we statistically compared performance of our five models using the logged runs both on training data (step 6 in Figure 2) and test data to choose the best performing model on unseen data (step 7 in Figure 2).

2.1.Development of the standard GA

The data we used for this project is a table with 588 rows/patients and 20 columns/features where the last column is the value we intend to predict. We split it into training and test data using a 90/10 split, resulting in n = 529 for training and n = 59 for testing. With m the number of features used to predict y, m = 19.

The standard GA we developed is very similar to the one developed during practical classes with some differences. One of them is the crossover and mutation operators. Since individuals in our algorithm can be interpreted as points in an m-dimensional space (with m being the number of features), we decided to use geometric crossover and geometric mutation (Figure 3).

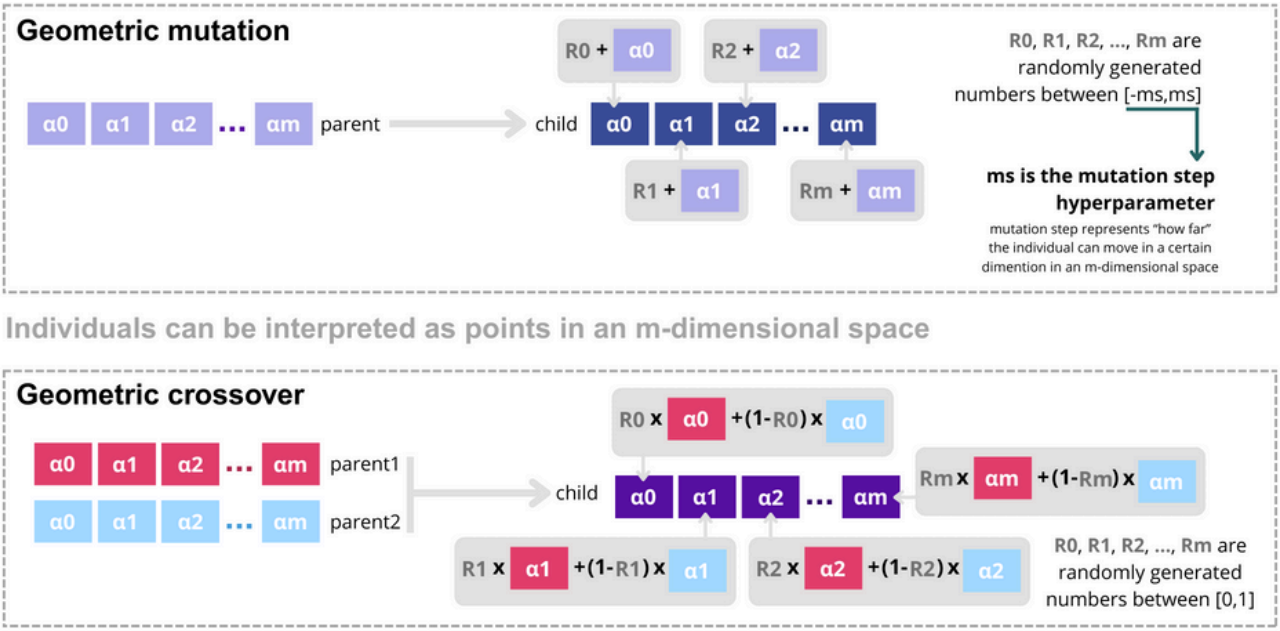

Figure 3. Geometric operators. The top panel: geometric mutation – taking 1 parent and generating a child by adding random modifications in the range [- mutation step, mutation step] to each coefficient. The bottom panel: geometric crossover – taking two parents and generating a child by creating a weighted average of both parents for each coefficient, where the weights are generated randomly between 0 and 1.

Geometric mutation works by moving the parent individual (a point in an m-dimensional space) a little bit in every dimension to create the child individual. How much the individual moves in each dimension is controlled by randomly generated numbers whitin the interval [- mutation step, mutation step]. Mutation step is, therefore, the evolution hyperparameter that controls how drastic the geometric mutation operation is (see Figure 3, top panel).

Geometric crossover works by creating a child that is geometrically between the two parents. To do this, a weighted average of the parents is calculated for each dimension where the weights are randomly selected from the interval [0,1] for each dimension (see Figure 3, bottom panel).

Another difference from the GA developed in class is that the mutation step hyperparameter is updated after a certain number of generations in a mechanism inspired by simulated annealing. Since in the beginning of the evolution process large mutations help to explore the solution space, it's useful to start out with a relatively large mutation step. However, after some generations, the algorithm should focus more on exploiting good solutions it has found to improve those. In our GAs, mutation step starts out large, stays large for a certain number of generations and, after that, mutation step gets progressively smaller by dividing it every generation by a certain value (the starting mutation step value, how many generations it goes without being updated as well as the value used to update mutation step are evolution hyperparameters).

Besides geometric operators and mutation step updating, our GA also starts the founder generation in a different way than was done in class due to the type of data being very different than the example we covered in class. To initialize the founder generation each run, we created individuals by doing the following: for the intercept (α0) generate a random number between min(all features) and max(all features). For each of the other coefficients that correspond to a specific feature (α1 to α19), they were chosen as a random value from between min(corresponding feature), max (corresponding feature), see Figure 4.

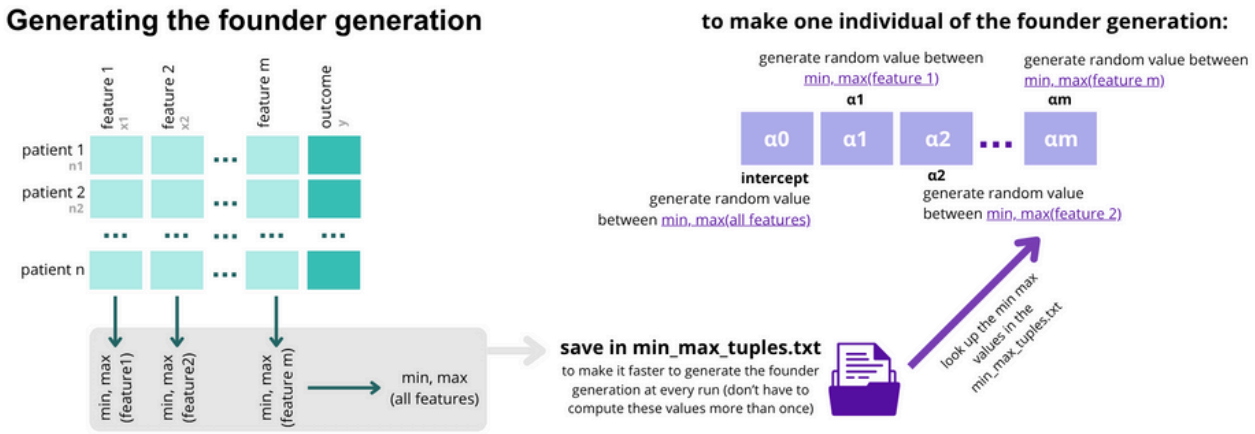

Figure 4. Generating the founder generation. Every run, a founder generation needs to be created. To make one individual of the founder generation, the min and max values for all features and for each feature are looked up in the min_max_tuples.txt file (this file is created and "writen in" on the first run). For the intercept a random value between min and max of all features is chosen and for all remaining coefficients, a value is chosen between the min and max of the corresponding feature.

Since this process is done thousands of times during the development, hyperparameter tuning and logging for each GA, we decided to create a txt file where these min, max tuples are saved for the whole data (needed to generate intercepts) and for each feature/column (needed to generate all other coefficients). In this way, these values do not have to be computed every run, saving time and computational efforts for the evolution process. This min_max_tuples.txt file is automatically used if

it exists and, if it does not, it's created on the first run and is used from then on.

The other two differences between our GA and the GA implemented in class are the way that fitness of each individual is evaluated and that we keep track of the diversity of each generation, displayed in Figure 5. The diversity of each generation is calculated by summing the variances of each coefficient; as previously mentioned, RMSE is the measure we use for fitness. RMSE reflects how "far off" ypred is from yreal across all patients when combining the clinical data and the coefficients that make up the individual.

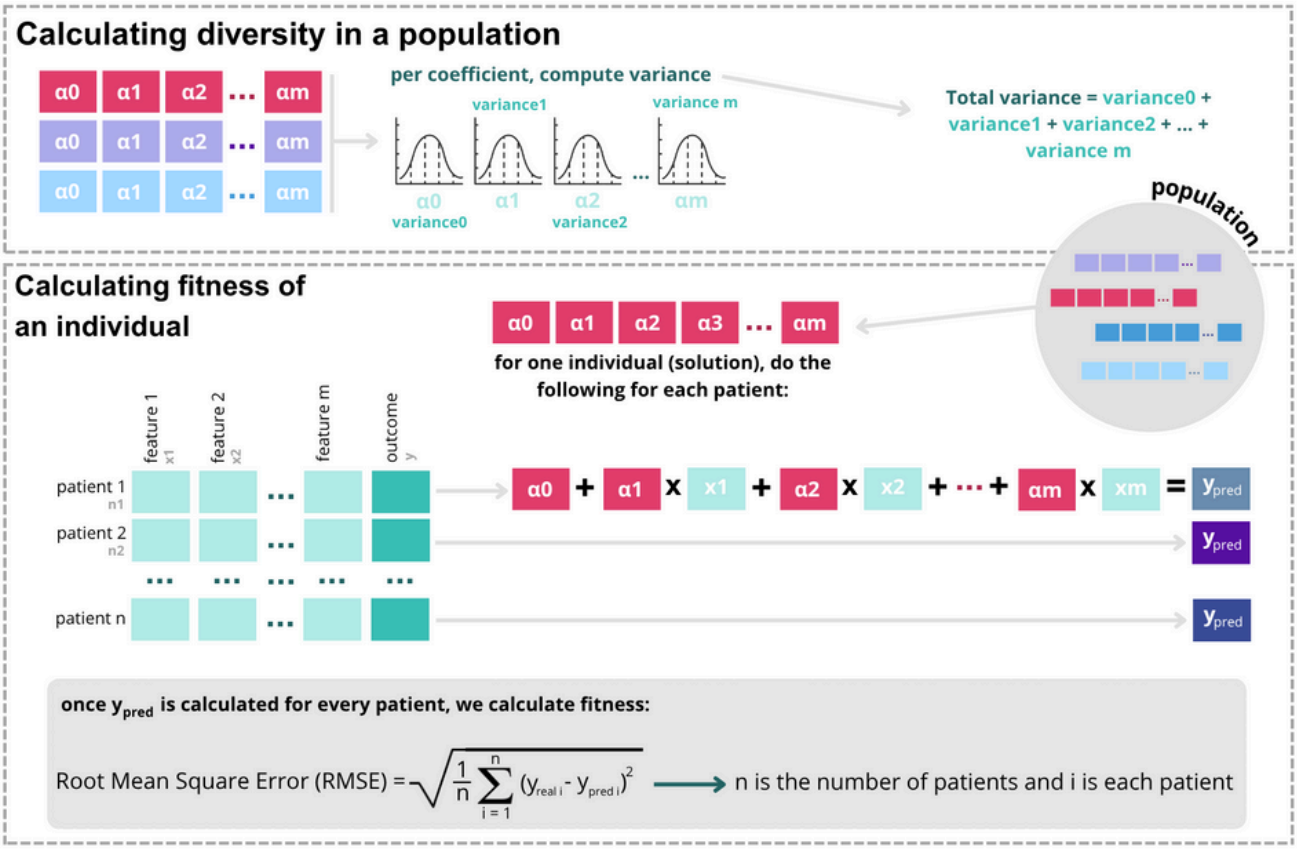

Figure 5. Calculating fitness and diversity. Top panel: calculating population diversity – the variance of each coefficient across individuals in the population is computed. These variance values are summed up to create a measure of population diversity. Bottom panel: calculating the fitness of an individual – calculating RMSE of an individual to reflect a measure of how right/wrong the individual is across all patients.

An overview of our standard GA is shown on Figure 6. Essentially, the founder generation is created, and its' fitness and diversity are computed and the generation is logged. Next, for n generations (n is a hyperparameter), evolution will take place. At each generation, the new generation is created in the form of an empty dictionary; the elite of the last generation is added to the new generation dictionary (elitism). A list is created to start saving the fitnesses of the elites (this is necessary for hyperparameter tuning later) and the generation starts being filled up/populated until the new generation has the intended number of individuals (population size is also a hyperparameter). To add one individual, an operator is chosen from three possibilities 1. geometric crossover (with a certain probability that is a hyperparameter), 2. "lucky bastard" in which the

parent passes to the next generation unchanged (with a certain probability that is also a hyperparameter) and 3. geometric mutation (with the probability 1- (P(GC) + P(GM))).

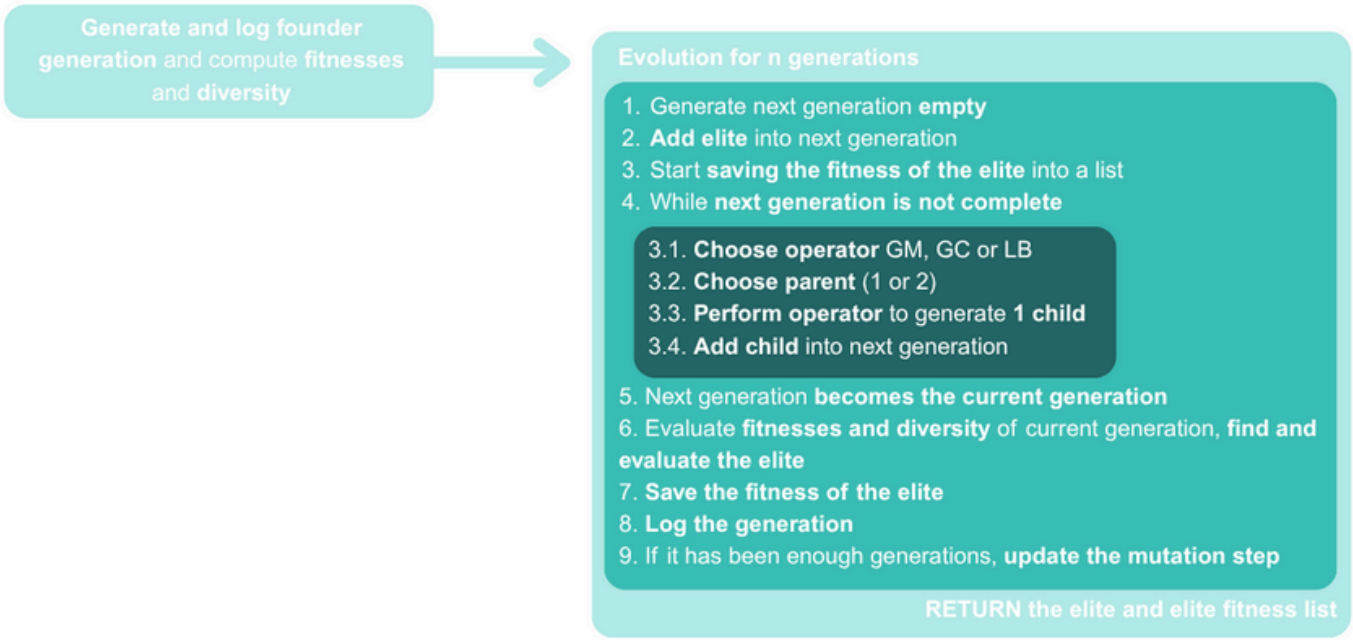

Figure 6. Overview of standard GA.

Next one or two parents (depends on the chosen operator) are chosen via tournament selection, with a given tournament size that is a hyperparameter. The operator is applied to the parent(s) to generate one child that is added into the generation dictionary where the key is an id structured id_generation_individual and the value is the individual's genotype (list of coefficients).

This "next generation" becomes the current generation and it's evaluated and fitnesses are saved in a dictionary where the keys are the individual ids and the values are the RMSE (phenotype). Generation number, elite individual (in the format [id, genotype, phenotype]) and population diversity are logged into the run log txt file. Elite fitness is saved into the elite fitness list. If it has been enough generations, the mutation step is updated. After all generations are done, the last generation elite and the elite fitness list are returned.

2.2.Development of the extinction GA

The version of our GA with extinctions functions in the same way as the standard GA when it comes to starting the founder generation,selecting parents and operators, the operators that can be selected, logging, update of the mutation step and fitness and diversity calculations. An overview of the extinction GA is shown on Figure 7, where the differences between the extinction GA and standard GA are apparent.

In this version of the algorithm, evolution happens for the same n generations, where the first x generations (x is a hyperparameter) don't have extinction available and so, for those x generations, evolution happens in the same way as the standard GA. When x+1 generation is reached, extinction events become a possibility. Every 20 generations a list of the fitness of the 20 most recent past elites is evaluated to see if fitness has stagnated in those 20 generations. If the difference between the max

and fitness of those 20 most recent elites is less than max_delta, a hyperparameter, we consider that fitness has stagnated, and an extinction event is triggered.

The next generation will be an extinction generation and its' population size will be vastly reduced (the extinction population size is a hyperparameter). Let's say the extinction generation is reduced to 15% of the size of the original population, those 15% are split among three categories of individuals that will be added to the extinction generation – for an egalitarian repopulation 5% elites, 5% "weirdos" which are the individuals most distant from the others in an m-dimensional space (how these individuals are selected will be covered later) and 5% immigrants which are individuals generated in the same way that the individuals in the founder generation are generated. The percentages taken up by each kind of survivor are also hyperparameters.

Tournament size is shrinked to relax selective pressure for the next c generations (relaxed tournament size is a hyperparameter as well as r, the number of relaxed selective pressure generations) and extinction is made unavailable for the next s generations (s is the number of safe generations, a hyperparameter as well). Lastly, the list of elite fitnesses used to evaluate stagnation is wiped clean.

After this, evolution resumes as happens for the first x generations until a new extinction event is triggered. After n generations, the return is the elite of the last generation in the format [id, genotype, phenotype].

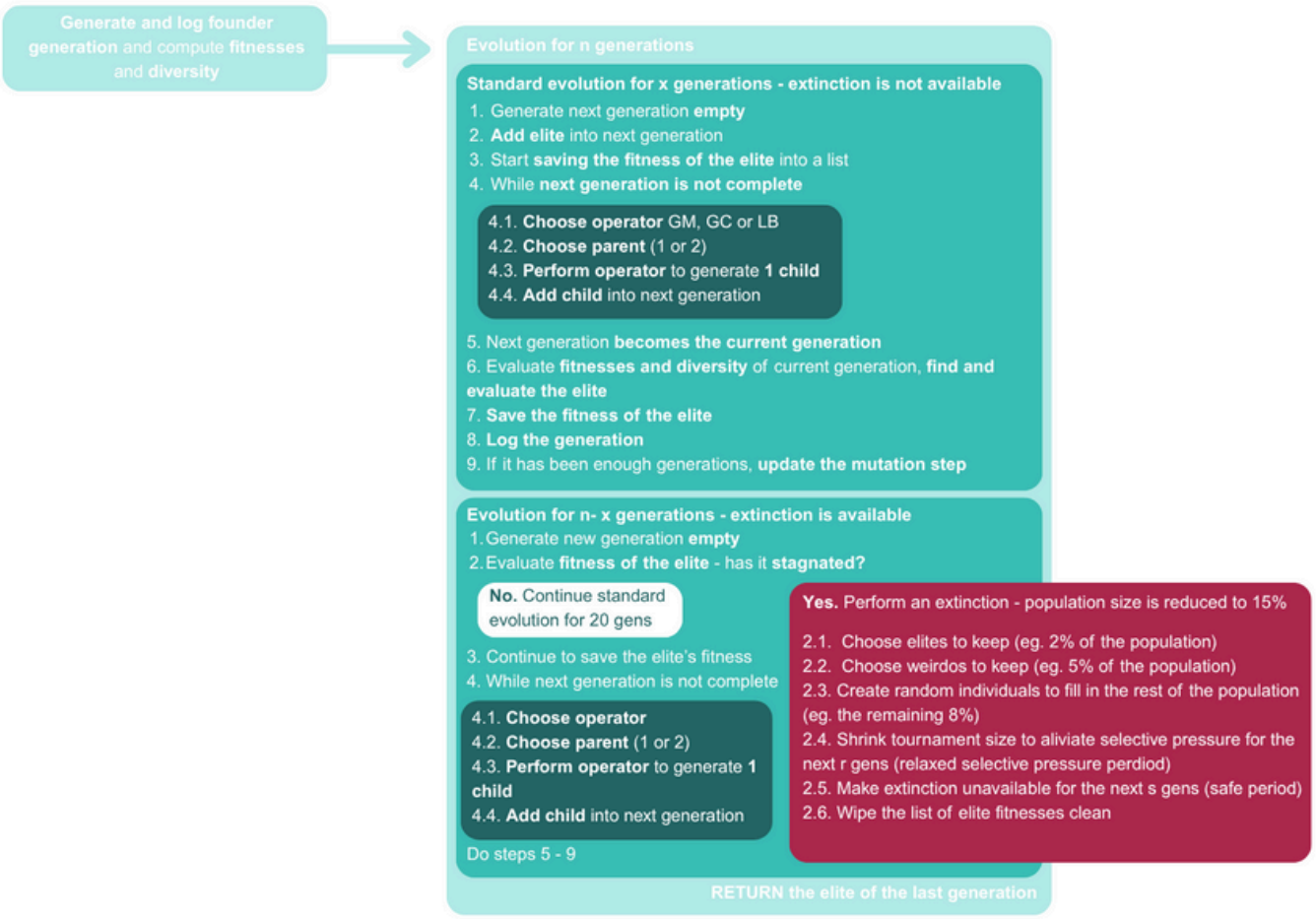

Figure 7. Overview of extinction GA algorithm.

"Weirdo" individuals and immigrants are types of individuals that were not a part of the standard GA (at least explicitly and with such a important role). The idea behind making "weirdo" and immigrants an important part of repopulation after extinction is to force the algorithm to do some exploring when fitness is stagnated by making it more likely for 1. "weirdos" and immigrants to be parents by relaxing tournament size and 2. for elites to reproduce via crossover with individuals that are different from themselves. Remember that geometric crossover creates children that are somewhere between the parents in an m-dimensional space so, in theory, by doing crossover with elites and individuals somewhat far away, exploring new areas of the search space is encouraged. If on one hand the goal of extinction is to encourage exploration, on another hand, it's important to not lose the elites we already so some elites are always integrated in the extinction generation.

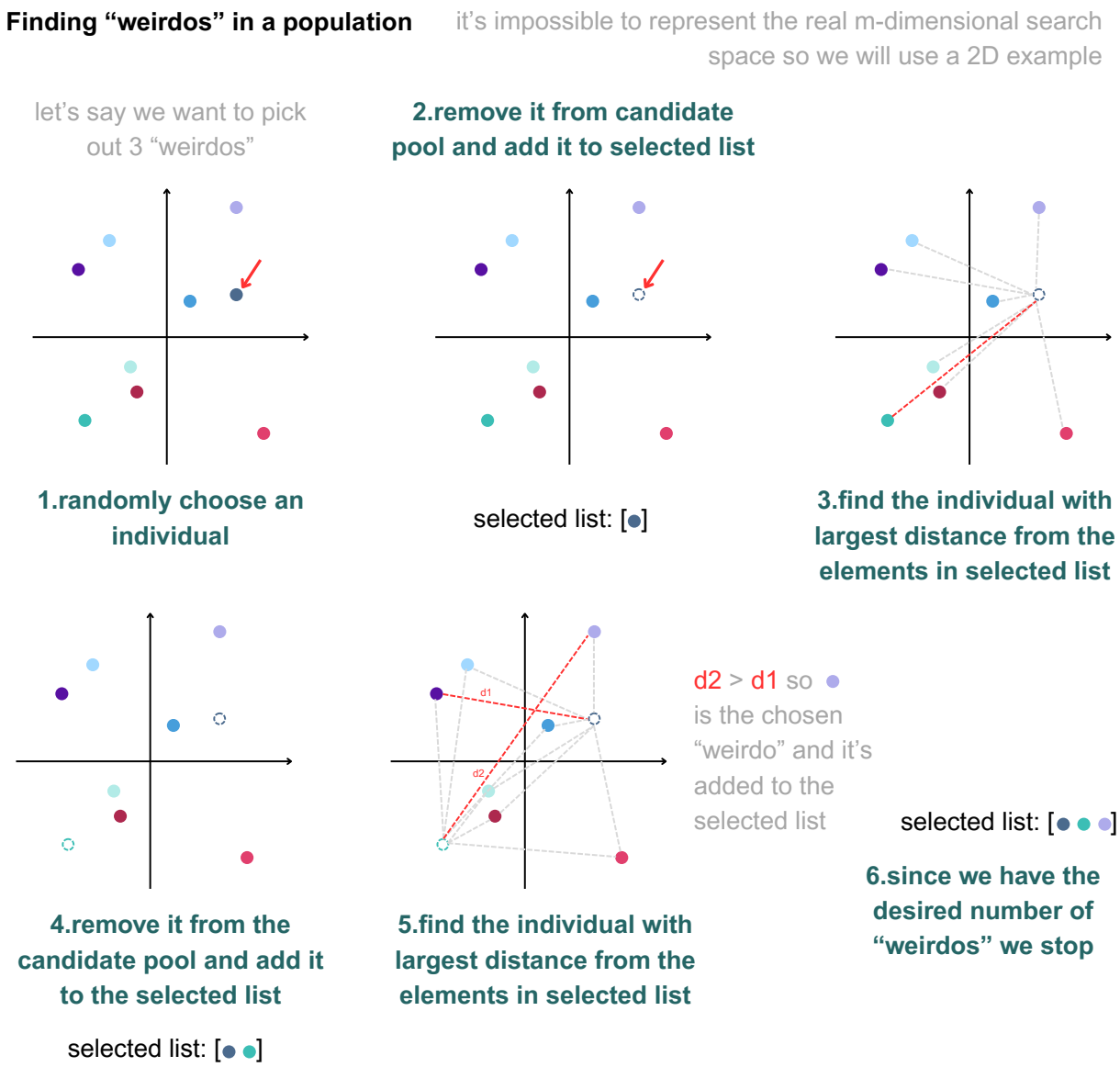

Figure 8. How we find "weirdos" in a population using Euclidian distances in the m-dimensional space where individuals can be represented as points .

“Weirdo” individuals are, as elites, present in the generation before the extinction generation (GBEG) and we find them via the mechanism shown in Figure 8. From the GBEG, we choose a random individual and remove it from the list of ids and add it to the list selected. Next, we fill the list one by one until we have the desired number of "weirdos". To add one element to the list we assume that every individual still on the list of ids is a candidate and compute its’ Euclidian distance to the points currently in the list selected. The individual with the largest distance to the group is selected as a “weirdo” and is added to the selected list.

In this way, "weirdos" are a sample of the most different individuals in the GBEG. Immigrant individuals are not a part of the GBEG at all and are created in the same way that founder individuals are created. The variability that they introduce is much larger than the variability introduced by "weirdos" and can be used, via crossover, to cross long distances in the search space which can be useful if the local optima that we have landed on before extinction is not particularly good.

2.3. Hyperparameter tuning using Optuna

To do hyperparameter tuning we decided to use Optuna, a python library specifically developed for this task. The strategy employed by Optuna is to dynamically search the solution space for solutions that maximize an objetive defined by the user.

Optuna uses a form of Bayesian optimization, working by iteratively selecting hyperparameter combinations, evaluating their performance and learning from previous trials/runs to guide future combinations of hyperparameters that it tries out. Tree-structured Parzen Estimator (TPE) is used to find promising regions in the search space and early pruning strategies are used to stop the algorithm from going into “unproductive” regions in the search space (Akiba *et al*, 2019).

Another advantage of Optuna is that we can choose what we want to optimize toward. For this project we designed a measure that values obtaining a good (low) RMSE but mostly values steady evolution to choose the combination of hyperparameters that minimizes the following score:

$$\text{Score} = 0.3 \cdot \text{NormalizedFinalFitness} + 0.7 \cdot (-\text{NormalizedImprovementCount})$$

This score is a property of each combination of hyperparameters/property of each Optuna trial. The improvement count corresponds to the number of generations where fitness improved. Before calculating the score, we normalize the final elite fitness obtained with the hyperparameter combination by dividing it by 100 (because the final elite RMSE is always under 100) and normalize the improvement count by dividing it by the number of generations, making the two values fit between 0 and 1.

By using this score, we prioritize a combination of hyperparameters that makes for steady evolution and, although some weight is given to finding a good RMSE, we didn’t chose that as a priority because i) the same combination of parameters found by Optuna that gets the lowest RMSE might not be the best across runs when we run it more than once and ii) the end goal of this

project is to suggest a GA model that performs well both in training and test data and prioritizing RMSE could result in overfitting and poor performance in the test data.

2.3.1. Evolution hyperparameter tuning using Optuna

To tune the evolution hyperparameters population size, tournament size, crossover probability, mutation step, update mutation step, update mutation step after x gens and probability of being a "lucky bastard" we did 50 Optuna trials using sensible intervals for each hyperparameter. 300 generations were always the limit number of generations we ran all GAs for to be able to do a lot of runs and obtain plenty of data for each GA model in a timely manner.

The best combination of hyperparameters found was: population size = 768, tournament size = 19, crossover probability = 0.83266628601947, mutation step = 464, update mutation step = 1.0907061943267877, update mutation step after x gens = 50 and probability of being a "lucky bastard" = 0.15660853661618737. These evolution hyperparameters were used for all models from here on out.

2. 3. 2. Extinction hyperparameter tuning using Optuna

To tune the extinction hyperparameters extinction available after x gens, extinction generation size (in percentage of population size), number of cooldown/safe generations, relaxed tournament size and number of generations with relaxed tournament size after extinction generation, we did the same as described above for evolution parameters.

The best combination of hyperparameters found was extinction available after x gens = 69, extinction generation size (in percentage of population size) = 0.2849567979136528 number of cooldown/safe generations = 42, relaxed tournament size = 8, number of generations with relaxed tournament size after extinction = 28. These hyperparameters were tuned using egalitarian repopulation and will be used in the other three extinction GAs.

2.4.Running every model 10 times

After identifying a good set of hyperparameters, we ran each of the 5 GAs 10 times, logging each run into a txt file. For the extinction-based GA, we ran the different versions by manually adjusting two key hyperparameters: the elite percentage and the "weirdo" percentage in the extinction generation. The immigrant percentage was implicitly determined by subtracting the elite and weirdo percentages from 100%. We tested four repopulation strategies:

- Egalitarian model: 33% elite, 33% weirdos, 33% immigrants.
- Elite-led model: 50% elite, 25% weirdos, 25% immigrants.
- Weirdo-led model: 50% weirdos, 25% elite, 25% immigrants.
- Immigrant-led model: 50% immigrants, 25% elite, 25% weirdos.

2.5.Assessing model performance on training data

To analyze and compare the performance of the 5 models on training data, we produced plots detailing progression of elite fitness across generations and population diversity across generations

(in the results section) and did some statistical tests to assess possible differences in result quality by comparing the fitness of the last elites for the various models, considering the 10 runs per model. We did Kruskal-Wallis test with all 5 models as well as pairwise Mann-Whitney U tests for all possible combinations of pairs within the 5 models. Performance comparison on training data is discussed in the Results and Discussion section.

2.6.Assessing model performance on test (unseen) data

To know how each model performs on unseen data, we first had to get a median last elite individual from each model by finding the median of each coefficient across the runs of the same model.Using this median last elite genotype, we scored RMSE on test data (n=59) which is 10% of the original data given to us. Performance comparison on test data is discussed in the Results and Discussion section.

2.7.A random approach for alternative hyperparameter tuning

In addition to the 5 GAs that we tested, we decided to test another approach for hyperparameter tuning, particularly for the standard GA. For this, we set a somewhat wide range for each of the parameters as follows: population sizes = [50, 100, 200, 300, 500, 750, 1000], tournament size = [3, 4, 5, 6, 7, 8, 9, 10], crossover probability = [0.5, 0.6, 0.7, 0.8, 0.9], mutation step = [0.01, 0.1, 10, 100, 1000, 10000], update mutation step = [1.01, 1.05, 1.1, 2, 5, 10], update step after x gens = [100, 150, 200] and probability of being a “lucky bastard” = 0.1, for a total of 500 generations.

For each hyperparameter, here follows the justification for the ranges chosen:

- **Population size:** Large populations allow for higher genetic diversity and exploration capabilities, while smaller populations reduce computational cost with the risk of premature convergence. Typical ranges tested in literature are 30-1000. (De Jong & Spears, 1990; Grefenstette, 1986; Sipper *et al.* ,2017)
- **Tournament size:** Larger values increase selective pressure, accelarating convergence while smaller values do the opposite, maintaining diversity by giving the less fitted individuals a chance to reproduce the key is to maintain a sustainable balance of exploration and exploitation. Ranges tested in literature are usually 0.1% to 1% of population size (Sipper *et al.* ,2017).
- **Crossover probability:** Higher values of crossover probabilty allow for better genetic material recombination, and more more thorough exploration of the search space. Values tested in literature range between 0.6 and 0.9. (Grefenstette, 1986; De Jong & Spears, 1990)
- **Mutation step:** This hyperparameter affects the magnitude of change from parent to child in in an m-dimentional spcae (Doerr & Doerr, 2015) . Values are largely problem dependent so we proposed different orders of magnitude to be randomly tried.
- **Update mutation step:** The higher the value of this hyperparameter, the faster mutation step decays, the closer to 1, the slower the decay is. We proposed a range from 1.01 to 10.

- **Update mutation step after x gens:** This hyperparameter controls when mutation step decay starts, given that we ran the alternative standard GAs for 500 generations, we proposed values from 100 to 200, ensuring that that evolution goes on long enough for this decay to take place for some generation and show its' effects.
- **Probability of being a "lucky bastard":** This hyperparameter controls the probability that an individual is included in the next generation regardless of its' fitness, helping to keep diversity high.

With the hyperparameters ranges established, we did 1204 runs in which all of the hyperparameters were chosen at random, allowing for a large array of parameter combinations. From these 1204 randomly chosen combinations of hyperparameters, we chose only the top 23 best performing combinations of hyperparameters by setting a cutoff of RMSE = 10 for the returned solution, to proceed for the next step, since choosing more would be too time consuming. Each of these 23 GAs (with different hyperparameters), was run 10 times, to assure statistical significance. After this, the two top performing across the 10 runs were selected to be alternative standard GA 1 and alternative standard GA 2. These two models were compared in terms of performance to the standard GA with "Optuna hyperparameters" and the results for that are in the results section.

Alternative standard GA 1 had the following hyperparameters: population size = 1000, crossover probability = 0.7, mutation step = 100, tournament size = 7, probability of being a lucky bastard = 0.15, update mutation step = 5, update mutation step after x generations = 150. And alternative standard GA 2 had the following hyperparameters: population size = 1000, crossover probability = 0.8, mutation step = 1000, tournament size = 4, probability of being a "lucky bastard" = 0.15, update mutation step = 5, update mutation step after x generations = 150. They essentially differ in terms of crossover probability and tournament size.

3.Results and Discussion

In this section of the report, we will discuss and compare the performance on training data of the 5 GAs we developed using plots and results of statistical analysis. Furthermore, we will also explore how models do on unseen/test data.

3.1. There are no statistically significant differences between fitness of the returned solution across the five GAs – comparison of model performance on training data

The Kruskal–Wallis test ($p < 0.001$), followed by pairwise Mann–Whitney U comparisons (FDR < 0.05), revealed that the RMSE of solutions returned by the standard GA is significantly lower than those returned by the extinction-based GAs across the 10 runs. This something we can observe on Figure 9 top panel, where we see that the standard GA returns solutions around RMSE = 10 and the others return very similar solutions in terms of fitness around RMSE = 20. Furthermore, no statistical differences were found between the solutions returned by the four GAs with extinctions.

However, when we observe elite fitness progression as a whole, we see that models with extinctions achieve lower RMSE faster even before generation 50. This effect, however, cannot be attributed to

the extinction mechanism because extinctions are only made available after generation 69, and it probably is a result of a better initial population (e.g. in terms of fitness or diversity).

On one hand, RMSE drop is slower in the standard GA, achieving lower final RMSE. On another hand, RMSE drop is steeper between generations 80 and 120 but stagnates at a worse (higher) RMSE level on GAs with extinctions.

When it comes to population diversity (see Figure 9 bottom panel), we can see that the main factor responsible for diminishing diversity is update of mutation step that starts after 50 generations. Interestingly, the initial population for the extinction GAs seems to be more diverse which might be responsible for its success (steep drop of RMSE) before generation 50.

The extinction mechanism does not seem to be impacting the elite fitness progression, although it causes peaks of population diversity. This indicates that the diversity being introduced by the extinction mechanism, via “weirdos” and immigrants, is not productive likely because these tend to be high RMSE individuals and, therefore are not likely to reproduce/ be maintained in generations following extinction.

To make extinctions function better, we would have to focus on improving the extinction GAs to introduce useful novelty after extinctions instead of just novelty. This could be achieved by incorporating different kinds of mutations that are more directional and by implementing strategies that force the variable individuals to stick around in the population longer than just the extinction generation or the couple of generations after the extinction generation.

3.2. Standard GA has the best performance on unseen data – comparing performance of the five GAs on test (unseen) data

Using the “median last elites” generated from the last elites on the 10 runs of each model, we evaluated how good each of the five GA did on unseen data. The standard GA had a RMSE of 11.06 on unseen data, followed by the very similarly performing "weirdo” led repopulation GA and elite led repopulation GA with RMSEs of, respectively, 18.41 and 18.35. Next best was immigrant led repopulation with RMSE = 19.73 and the worst performing on test data was the egalitarian repopulation GA with RMSE = 23.31.

Upon analyzing the performance of the five GAs on training and test data, we can clearly conclude that the standard GA is the best performing with very little to no overfitting (very similar performance on training and test data).

3.3. The two alternative standard GAs generated through random hyperparameter tuning perform similarly on the training data but worse on the test data compared to the standard GA tuned with Optuna hyperparameters

As previously mentioned, out of the 5 initial models (1 standard GA and 4 extinction GAs), the best performing was the standard GA. This conclusion led us to focus on the standard GA and try out a different hyperparameter tuning strategy (described in section 2.7) to produce and test performance of two alternative standard GAs.

Similarly to what was done for the other 5 models, the performance of these standard GA alternative 1 and standard GA alternative 2 was assessed on training and test data, with a focus on

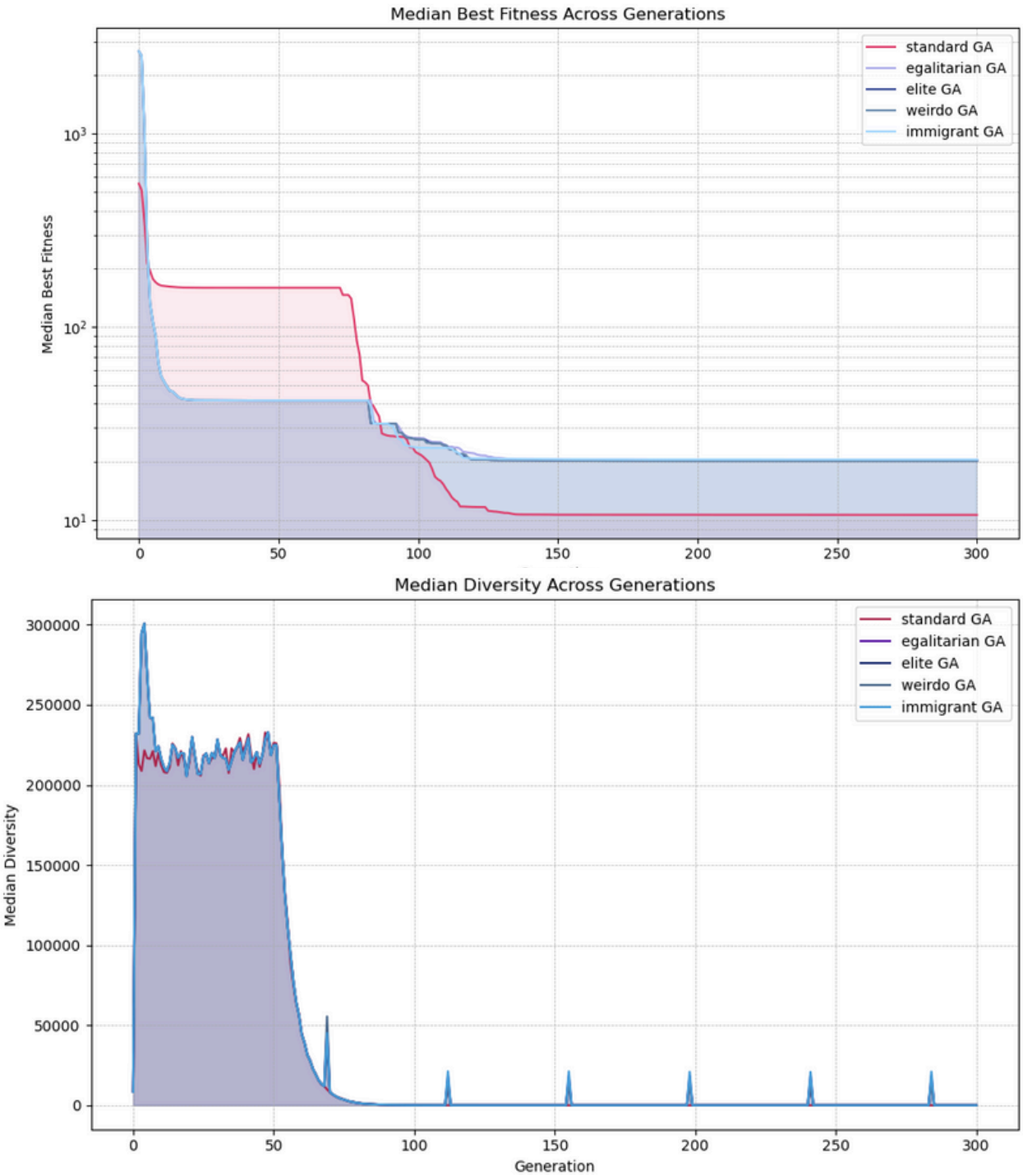

Figure 9. Comparison of model performance on training data across generations. Top panel: elite fitness across generations for the different models. Bottom panel: diversity measured by variance across generations for the different models.

As previously mentioned, out of the 5 initial models (1 standard GA and 4 extinction GAs), the best performing was the standard GA. This conclusion led us to focus on the standard GA and try out a different hyperparameter tuning strategy (described in section 2.7) to produce and test performance of two alternative standard GAs.

Similarly to what was done for the other 5 models, the performance of these standard GA alternative 1 and standard GA alternative 2 was assessed on training and test data, with a focus with comparing them to the standard GA with "Optuna hyperparameters".

One big difference between the alternative standard GAs and the standard GA is that they run for 500 generations instead of 300, something we can see in Figure 10, below. However, the extra 200 generations do not seem to be productive, as solutions across runs after 500 generations returned by the alternative standard GAs are not statistically different from the solutions returned after 300 generations by the standard GA (Kruskal-Wallis p-value = 0.1709). We can see that there's not much of a difference in Figure 10 between the final RMSE values achived by the 3 standard GAs, they all seem to be slightly above 10.

When assessing the performance of the alternative standard GAs on test data, we found that they do worse than the standard GA with "Optuna hyperparameters", with alternative standard GA 1 having an RMSE of 13.32 and alternative standard GA 2 having an RMSE of 20.53, making standard GA the undisputed winner in terms of performance out of the 5 original GAs as well as winning over the alternative standard GAs developed with random hyperparameter tuning.

This is somewhat expected as the strategy employed by Optuna for hyperparameter tuning is less naive than the random hyperparameter search done to give rise to the two alternative standard GAs. Furthermore, hyperparameters for the alternative standard GAs are quite different from the hyperparameters obtained by Optuna so this difference in performance is expected, especially on unseen data.

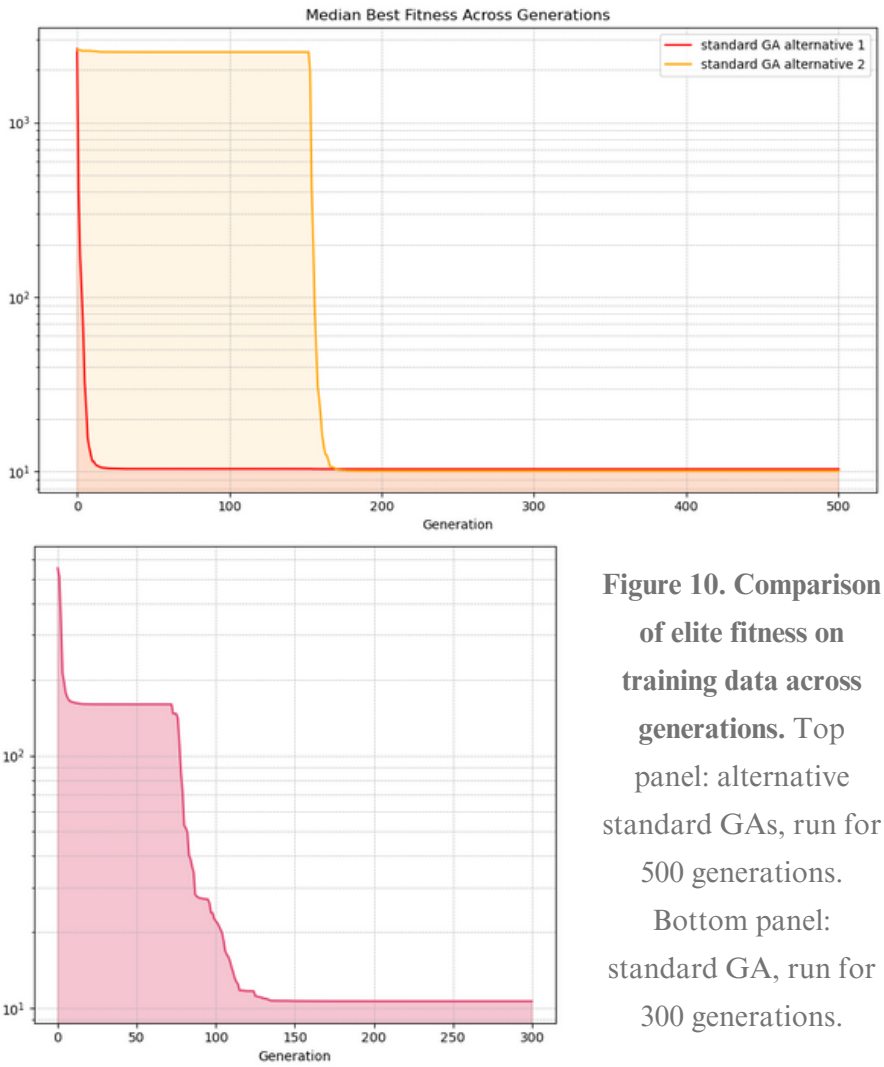

Figure 10. Comparison of elite fitness on training data across generations. Top panel: alternative standard GAs, run for 500 generations. Bottom panel: standard GA, run for 300 generations.

4.Conlcusion

In this project, we investigated the application of Genetic Algorithms (GAs) to optimize the coefficients of a linear regression model for predicting Parkinson's Disease symptom severity using UPDRS scores. First, we developed and compared five distinct GA models: a standard GA and four extinction-based variants with different repopulation strategies. Our goal was to assess whether extinction mechanisms could enhance exploration, avoid premature convergence, and improve predictive performance.

Based on our experimentation, the standard GA with Optuna-optimized hyperparameters emerged as the clear winner, achieving the lowest RMSE on both training and unseen test data (11.06 on test data), and showing minimal overfitting. The extinction-based GAs, although achieving faster initial RMSE drops, likely due to more diverse initial populations, ultimately stagnated at higher RMSE levels (~20 on training data) and underperformed on test data. The "weirdo"-led and elite-led extinction GAs performed best among extinction variants, but none surpassed the standard GA.

Extinction mechanisms did not improve elite fitness progression: although they produced peaks of population diversity, these were largely useless for evolution of better solutions. Diversity introduced via "weirdos" and immigrants was probably made up of high-RMSE individuals unlikely to survive in the generations following extinction. Therefore, extinction GAs failed to maintain or propagate useful novelty after extinction events, limiting their effectiveness.

To address this, future work could focus on making extinction mechanisms more effective by introducing useful novelty rather than random diversity. This can be achieved by incorporating directional mutations and designing mechanisms that ensure variable individuals persist across multiple generations, fostering meaningful exploration rather than quickly eliminating them.

Additionally, the two alternative standard GAs, produced using random hyperparameter tuning and run for 500 generations, did not outperform the standard GA. Despite running for more generations, their final elite fitness was statistically similar to the standard GA after 300 generations, and both performed worse on test data (RMSE of 13.32 and 20.53, respectively) than the standard GA. This confirms that not only a "non naive" hyperparameter tuning strategy (via Optuna) is important, but also that simply increasing the number of generations does not necessarily yield better generalization for this problem.

Exploring alternative Optuna optimization objectives or more advanced tuning strategies could further enhance the performance of both standard and extinction-based GAs. While extinction mechanisms remain promising for encouraging exploration, their current implementation requires refinement to realize their full potential.

In conclusion, for this regression task, the standard GA with Optuna-tuned hyperparameters remains the most robust and effective solution, and thus, is the GA we propose to solve this problem.

References

Kumar V, Chhabra JK, Kumar D. Parameter adaptive harmony search algorithm for unimodal and multimodal optimization problems. J Comput Sci. 2014;5(2):144–155.

Kumar V, Kumar D. An astrophysics-inspired grey wolf algorithm for numerical optimization and its application to engineering design problems. Adv Eng Softw. 2017;112:231–254.

Michalewicz Z. Genetic algorithms + data structures = evolution programs. New York: Springer-Verlag; 1992.

Kennedy J, Eberhart RC. Proceedings of IEEE international conference on neural networks (1995) 1995. Particle swarm optimization; pp. 1942–1948.

Dorigo M, Birattari M, Stutzle T. Ant colony optimization - artificial ants as a computational intelligence technique. IEEE Comput Intell Mag. 2006;1(2006):28–39.

Michalewicz Z, Schoenauer M. Evolutionary algorithms for constrained parameter optimization problems. Evol Comput. 1996;4(1):1–32. doi: 10.1162/evco.1999.7.1.1

Holland J.H. Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence. Complex Adaptive Systems, A Bradford Book; Cambridge, MA, USA: 1992.

Fogel D.B., Fogel L.J. An introduction to evolutionary programming. In: Alliot J.M., Lutton E., Ronald E., Schoenauer M., Snyers D., editors. Proceedings of the Artificial Evolution. Springer; Berlin/Heidelberg, Germany: 1996. pp. 21–33. Lecture Notes in Computer Science.

Burak J., Mengshoel O.J. A multi-objective genetic algorithm for jacket optimization; Proceedings of the Genetic and Evolutionary Computation Conference Companion; Lille, France. 10–14 July 2021; New York, NY, USA: Association for Computing Machinery; 2021. pp. 1549–1556. GECCO'21

Barbareschi M., Barone S., Bosio A., Han J., Traiola M. A Genetic-algorithm-based Approach to the Design of DCT Hardware Accelerators. ACM J. Emerg. Technol. Comput. Syst. 2022;18:50:1–50:25. doi: 10.1145/3501772.

Peng C., Wu X., Yuan W., Zhang X., Zhang Y., Li Y. MGRFE: Multilayer Recursive Feature Elimination Based on an Embedded Genetic Algorithm for Cancer Classification. IEEE/ACM Trans. Comput. Biol. Bioinform. 2021;18:621–632. doi: 10.1109/TCBB.2019.2921961.

Bonabeau E, Dorigo M, Theraulaz G. Swarm intelligence: from natural to artificial systems. Inc: Oxford University Press; 1999.

Ye, G., & Kang, D. (2021). Extended Evolutionary Algorithms with Stagnation-Based Extinction Protocol. Applied Sciences, 11, 3461. https://doi.org/10.3390/APP11083461.

W. Poewe, K. Seppi, C. M. Tanner, G. M. Halliday, P. Brundin, J. Volkmann, A.-E.Schrag, and A. E. Lang, "Parkinson disease," Nature reviews Disease primers, vol. 3,no. 1, pp. 1–21, 2017.

M. Castelli, L. Vanneschi, and S. Silva, "Prediction of the unified parkinson's diseaserating scale assessment using a genetic programming system with geometric semantic genetic operators," Expert Systems with Applications, vol. 41, no. 10, pp. 4608–4616, 2014.

Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. https://doi.org/10.1145/3292500.3330701.

De Jong, K.A., & Spears, W.M. (1990). An Analysis of the Interacting Roles of Population Size and Crossover in Genetic Algorithms. Proceedings of the First Workshop on Parallel Problem Solving from Nature, Springer-Verlag, Berlin, pp. 38–47.

Grefenstette, J.J. (1986). Optimization of Control Parameters for Genetic Algorithms. *IEEE Transactions on Systems, Man, and Cybernetics*, 16(1), 122–128.

Sipper, M., Fu, W., Ahuja, K., & Moore, J.H. (2017). Investigating the Parameter Space of Evolutionary Algorithms. *BioData Mining*, 10(1), 1–23.

Doerr, B., & Doerr, C. (2015). Optimal Parameter Choices Through Self-Adjustment: Applying the 1/5-th Rule in Discrete Settings. *arXiv preprint arXiv:1504.03212*.